

DISTRIBUTED JOB SCHEDULER

Backend Internship Assignment Report

Submitted By

Reg No

Krishna Rajaboina

RA2311028020023

Technology Stack: Node.js | Express.js | PostgreSQL | Prisma | React | JWT | Swagger

GitHub Repository:

<https://github.com/KrishnaRajaboina/Distributed-Job-Scheduler>

TABLE OF CONTENTS

1. Introduction
2. Project Objectives
3. Problem Statement
4. System Overview
5. Technology Stack
6. Software Architecture
7. ER Diagram
8. Project Features
9. Backend Implementation
10. Frontend Implementation
11. Database Design using Prisma
12. Authentication using JWT
13. REST API Documentation
14. Testing and Validation
15. Screenshots
16. Challenges Faced
17. Future Enhancements
18. GitHub Repository
19. Conclusion

1. Introduction

The Distributed Job Scheduler is a full-stack web application developed to automate and manage job execution across multiple workers and queues. The system enables users to create, schedule, execute, monitor, and manage background jobs efficiently. It supports worker management, queue management, scheduled execution, authentication, and real-time monitoring through a responsive dashboard.

The backend is built using Node.js and Express.js with PostgreSQL as the database managed through Prisma ORM. The frontend is developed using React and communicates with the backend using REST APIs.

2. Project Objectives

The primary objectives of this project are:

- Develop a scalable job scheduling system.
- Manage multiple workers efficiently.
- Implement queue-based job execution.
- Secure APIs using JWT authentication.
- Provide API documentation using Swagger.
- Store application data using PostgreSQL.
- Monitor jobs through a modern React dashboard.
- Demonstrate backend architecture and REST API development.

3. Problem Statement

Organizations often need to schedule repetitive and background tasks such as report generation, email notifications, backups, and data synchronization. Manual execution is inefficient and error-prone.

The Distributed Job Scheduler solves this problem by automatically scheduling and distributing jobs among available workers while maintaining queue management and monitoring capabilities.

4. System Overview

The application consists of two major components:

Backend

- Express.js REST API
- Prisma ORM
- PostgreSQL Database
- JWT Authentication
- Swagger Documentation
- Job Dispatcher Service
- Cron Scheduler

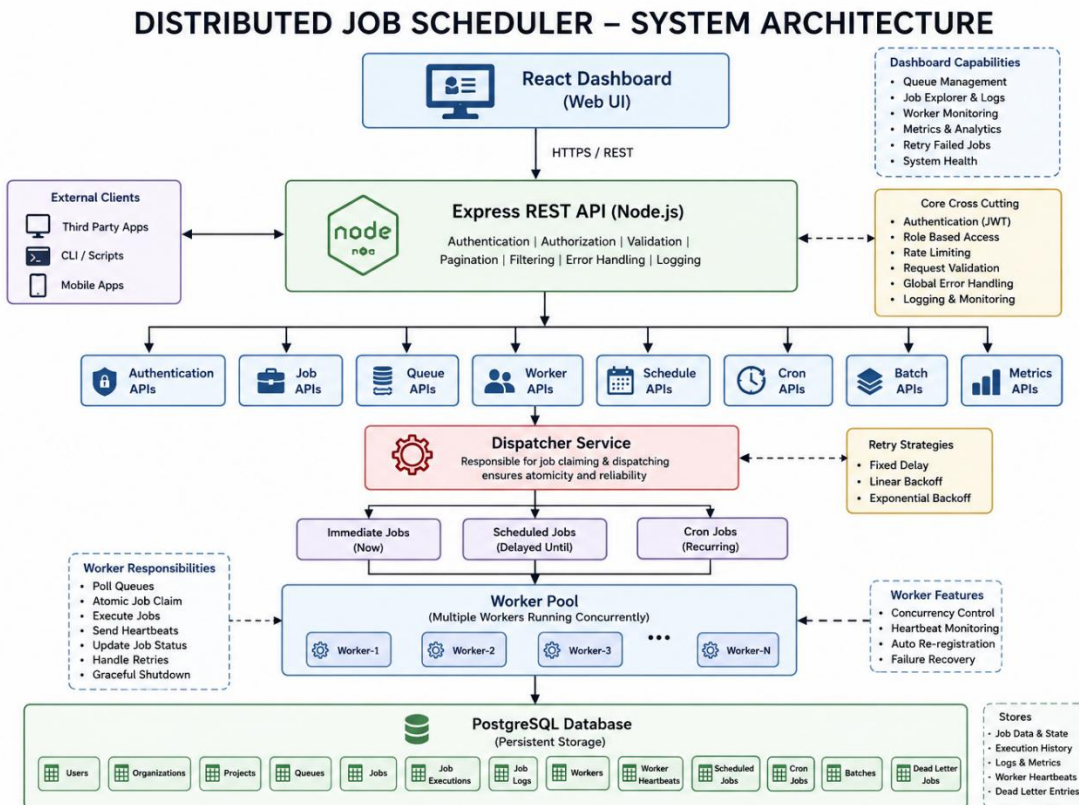
Frontend

- React
- Axios
- React Router
- Dashboard
- Worker Management
- Queue Management
- Job Management
- Metrics Dashboard

5. Technology Stack

Component	Technology
Frontend	React, Vite
Backend	Node.js, Express.js
Database	PostgreSQL
ORM	Prisma
Authentication	JWT
Documentation	Swagger
API Client	Axios
Version Control	Git & GitHub

6. Software Architecture

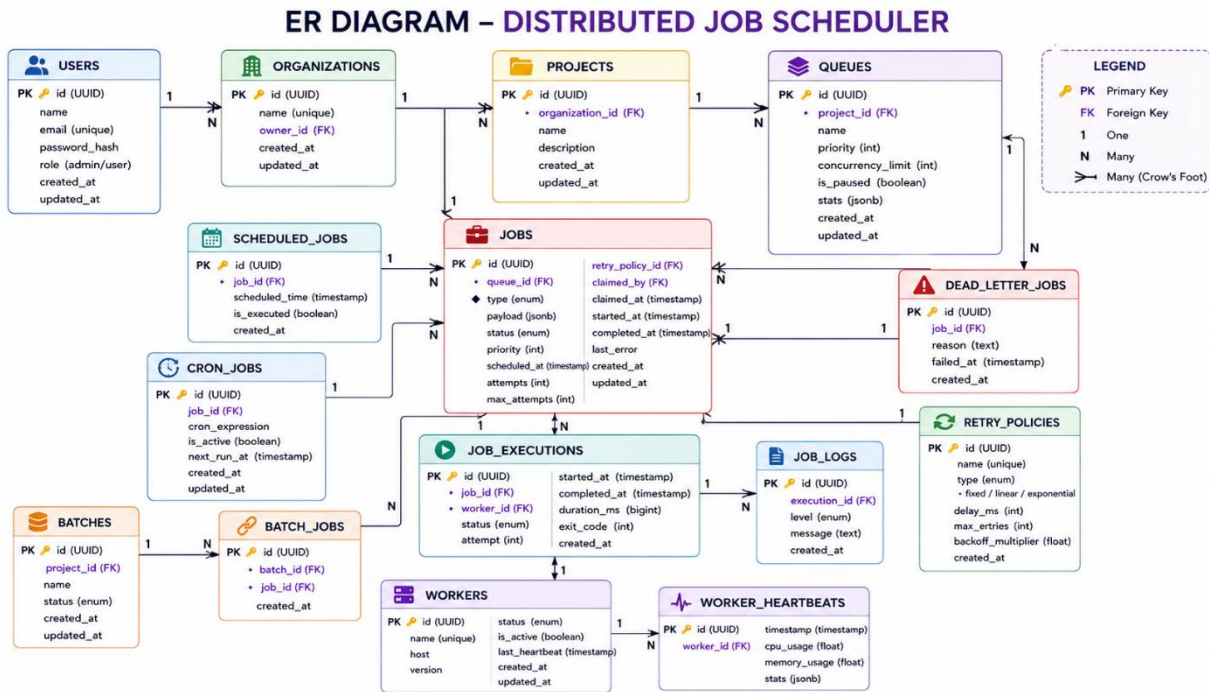


Architecture Description

The frontend communicates with the backend through REST APIs.

The backend processes requests using Express routes and controllers. Prisma ORM interacts with PostgreSQL to store and retrieve data. Worker services, queue services, scheduled jobs, and dispatcher services coordinate job execution. JWT authentication secures all protected APIs.

7. ER Diagram



Database Entities

- User
- Job
- Queue
- Worker
- Scheduled Job
- Batch Job

Relationships are maintained through Prisma models.

8. Project Features

Authentication

- User Registration
- User Login
- JWT Token Generation
- Protected APIs

Job Management

- Create Jobs
- Update Jobs
- Delete Jobs
- Monitor Job Status

Queue Management

- Create Queue
- View Queue
- Assign Jobs

Worker Management

- Register Workers
- Worker Heartbeat
- Worker Status Updates

Scheduler

- One-Time Scheduled Jobs
- Cron Jobs
- Batch Jobs

Metrics Dashboard

- Total Jobs
- Pending Jobs

- Running Jobs
- Completed Jobs
- Failed Jobs
- Active Workers
- Total Queues

9. Backend Implementation

The backend follows the MVC architecture.

Controllers

- Authentication Controller
- Job Controller
- Queue Controller
- Worker Controller
- Metrics Controller
- Scheduled Job Controller
- Batch Controller

Routes

- /api/auth
- /api/jobs
- /api/workers
- /api/queues
- /api/projects
- /api/metrics
- /api/scheduled-jobs
- /api/batch

Services

- Dispatcher Service
- Cron Service

10. Frontend Implementation

The frontend is built using React.

Pages

- Dashboard
- Jobs
- Workers
- Queues
- Metrics
- Login

Components

- Sidebar
- Navbar
- Dashboard Cards
- Layout

Axios is used to communicate with backend APIs.

11. Database Design using Prisma

Prisma ORM manages all database operations.

Advantages

- Type-safe queries
- Easy migrations
- Better schema management
- Automatic SQL generation

Commands used:

npx prisma generate

npx prisma migrate dev

npx prisma studio

12. Authentication using JWT

JWT authentication secures protected routes.

Authentication Flow

1. User logs in.
2. Server verifies credentials.
3. JWT token is generated.
4. Token is stored on the frontend.
5. Every API request includes:

Authorization: Bearer <token>

6. Backend validates the token before processing requests.

13. REST API Documentation

Swagger UI is integrated for API testing.

Swagger URL

<http://localhost:5000/api-docs>

Major APIs include

Authentication

- Register
- Login

Jobs

- Create Job
- Get Jobs

- Update Job
- Delete Job

Workers

- Register Worker
- Update Worker Status
- Heartbeat

Queues

- Create Queue
- Get Queue

Metrics

- Dashboard Metrics

Scheduled Jobs

- Create Scheduled Job
- Execute Scheduled Job

14. Testing and Validation

The project was tested using:

Swagger

Verified all REST APIs.

React Dashboard

Verified frontend integration.

Prisma Studio

Verified database records.

PostgreSQL

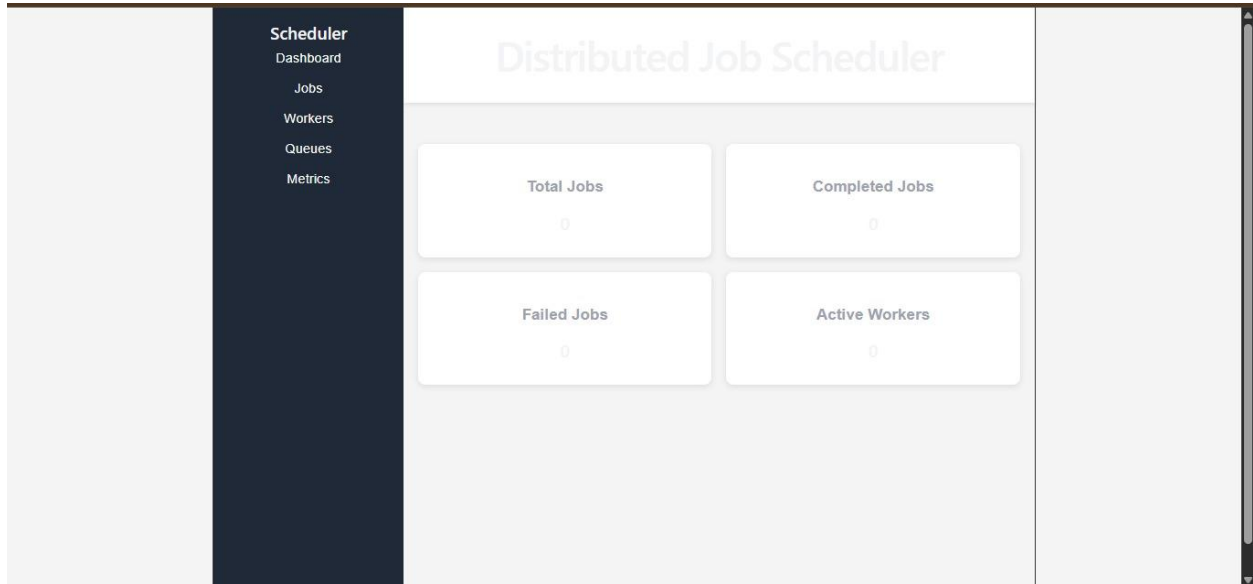
Verified table creation and relationships.

Authentication

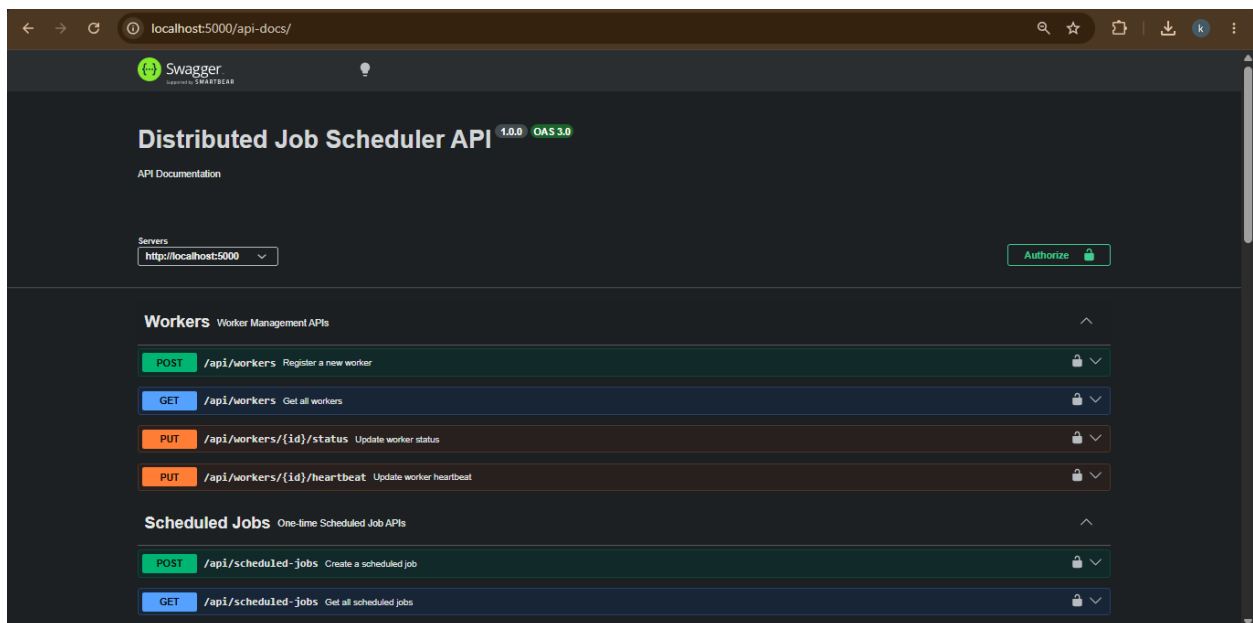
Verified JWT login and authorization.

15. Screenshots

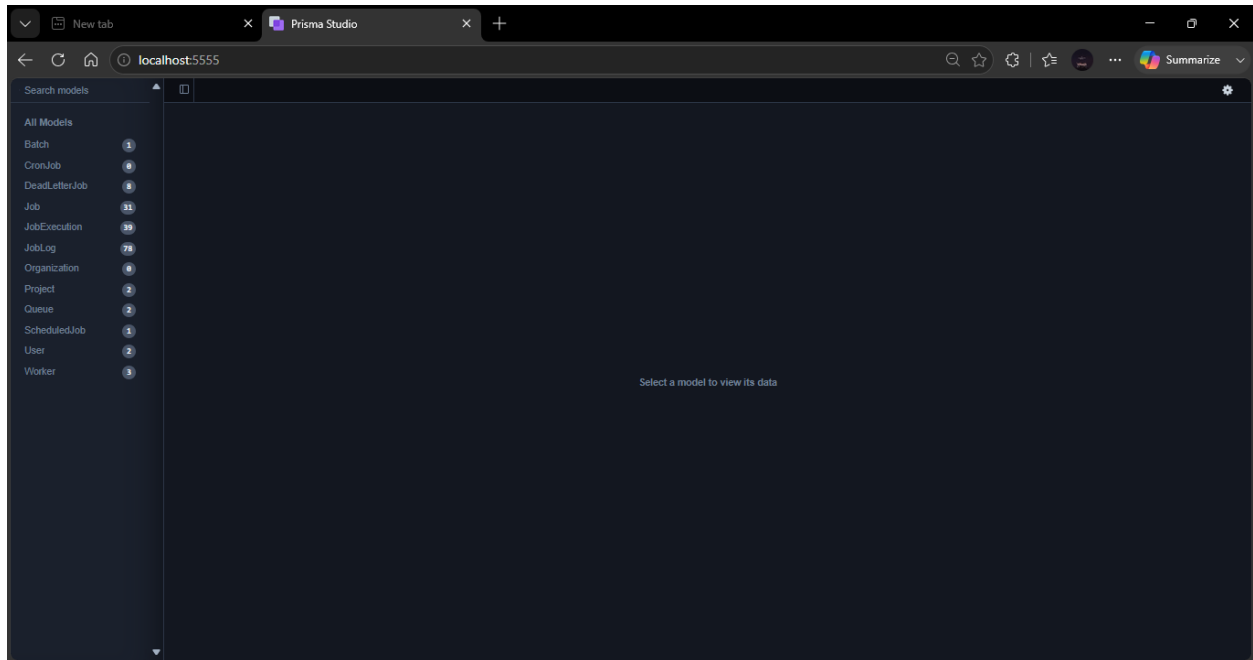
Dashboard



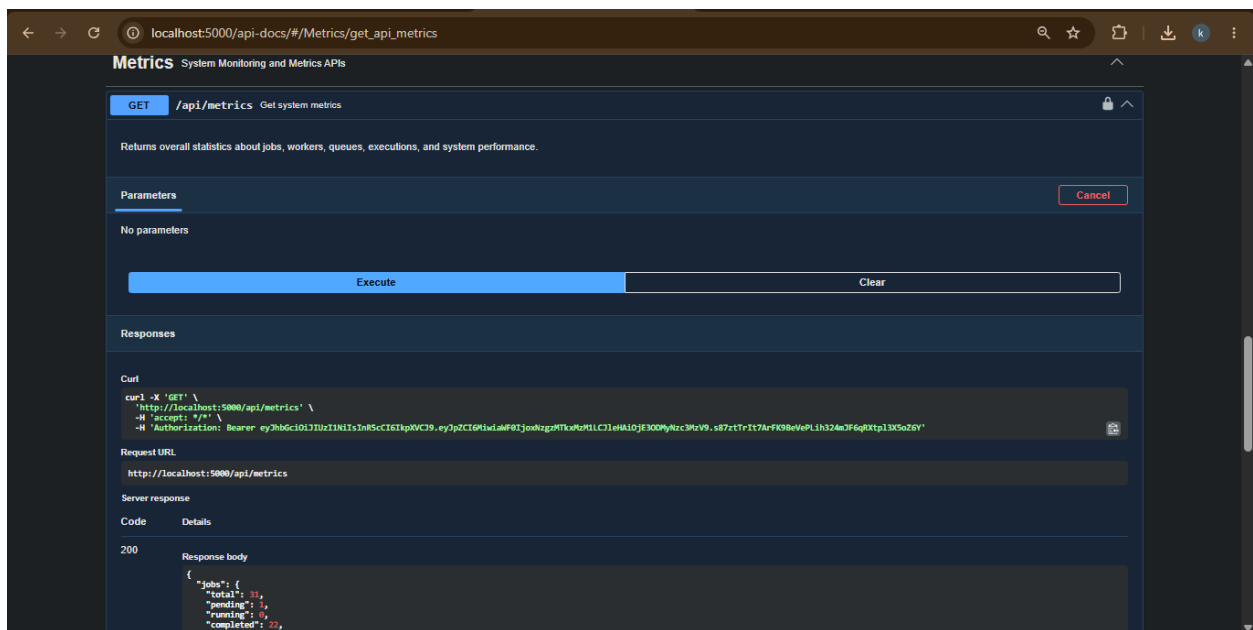
Swagger Documentation



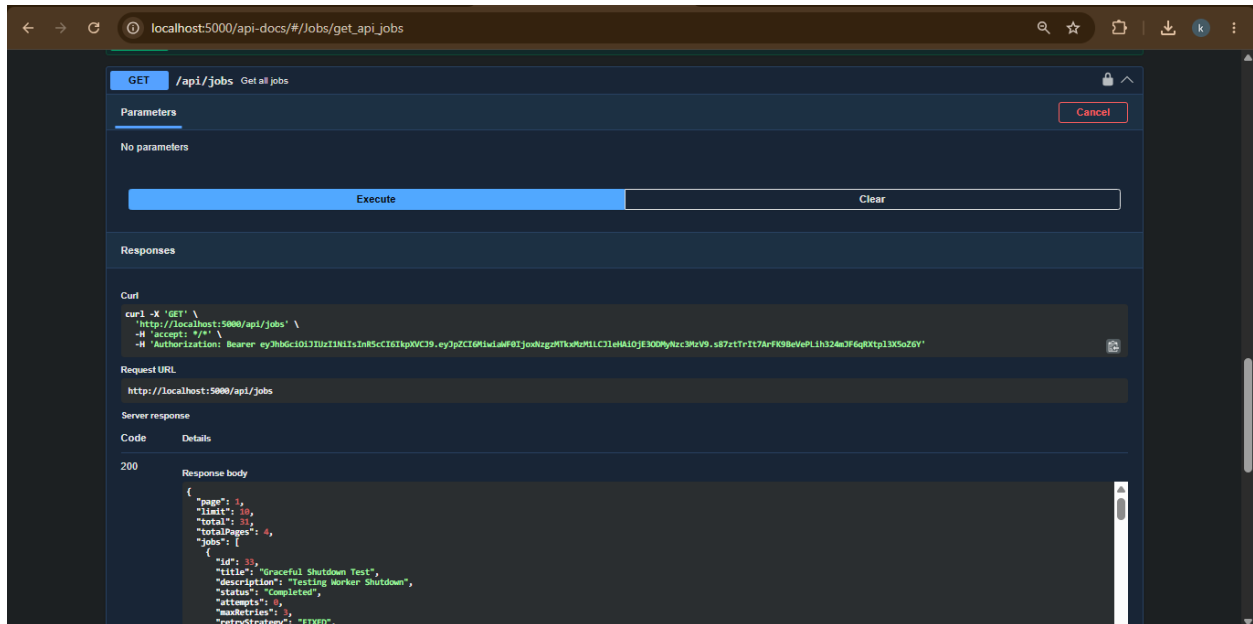
Prisma Studio



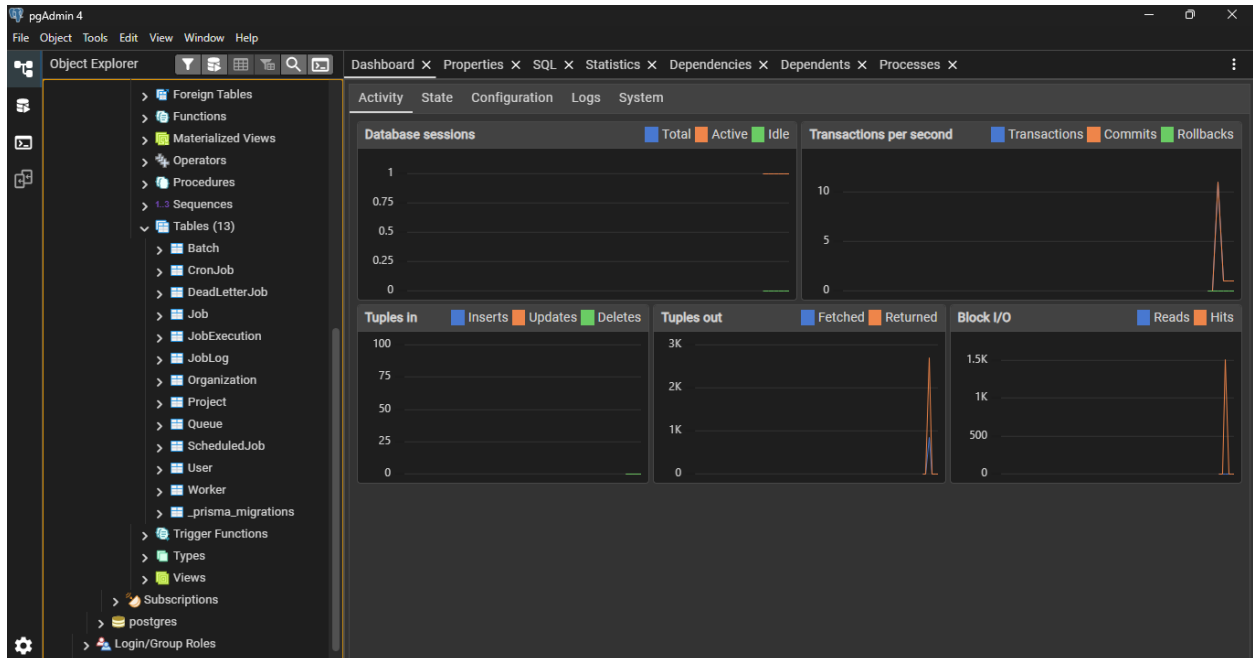
Metrics API



Jobs API



PostgreSQL Database



16. Challenges Faced

During the development of the project several challenges were encountered.

- JWT authentication issues
- Token validation errors
- React and backend integration
- API route management
- Prisma schema synchronization
- CORS configuration
- Dashboard metrics integration

All challenges were resolved through debugging, testing, and proper API validation.

17. Future Enhancements

The following features can be added in future versions.

- Email Notifications
- Redis Queue Integration
- Docker Deployment
- Kubernetes Deployment
- Role-Based Access Control
- Advanced Analytics Dashboard
- WebSocket Real-time Updates
- Cloud Deployment
- Job Retry Policies
- Notification System

18. GitHub Repository

Project Repository

<https://github.com/KrishnaRajaboina/Distributed-Job-Scheduler>

The repository contains:

- Backend Source Code
- Frontend Source Code
- Swagger Documentation
- Prisma Schema
- Design Documents
- Architecture Diagram
- ER Diagram
- Screenshots

19. Conclusion

The Distributed Job Scheduler successfully demonstrates modern backend application development using Node.js, Express.js, PostgreSQL, Prisma ORM, JWT Authentication, Swagger API Documentation, and React. The project provides a scalable architecture for scheduling, managing, and monitoring jobs through queues and workers while maintaining secure communication between frontend and backend.

The application satisfies the internship assignment requirements by implementing secure authentication, RESTful APIs, database management, API documentation, and an interactive dashboard. It also serves as a strong foundation for enterprise-level job scheduling systems and can be extended with cloud deployment, message queues, and real-time monitoring in future versions.